EAST WEST UNIVERSITY

Lab 5

Clustering YouTube Comments

Course Title: *Data Mining*

Course Code: *CSE 477*

Semester: *Fall 2025*

Section: *01*

Submitted by

Name: *Nushrat Jaben Aurnima*

ID: *2022-2-60-146*

Date: *26/11/2025*

Submitted to

Amit Mandal

Lecturer, Department of Computer Science and Engineering

East West University

Table of Contents

Introduction

This week's lab ***Clustering YouTube Comments***, we continued to work on the cleaned dataset created in Lab 2. The focus of this lab was to apply unsupervised learning techniques, K-Means and DBSCAN for finding hidden patterns in the text. This lab work introduces methods for grouping the comments based on their content. Working with real YouTube comments often implies dealing with short, informal, and sometimes inconsistent language. Therefore, this lab emphasized how to transform raw text into meaningful numerical features using TF-IDF, and clustering algorithms that behave differently depending on the structure of the data. Overall, the aim is to gain experience in understanding unsupervised models identify patterns in natural language and how to interpret those results meaningfully.

Objectives

- Apply TF-IDF vectorization on cleaned YouTube comments to convert numerical features that is suitable for clustering.
- Implement and analyze K-Means clusters using the elbow method to select optimal *k*.
- Run DBSCAN using different values of epsilon and observe density-based clustering on noisy handles irregular text patterns.
- Comparing the strengths and limitations of K-Means and DBSCAN when applied to short, informal social media comments.
- Implement PCA for cluster visualization and convey the major themes of each cluster.
- Develop a competent understanding of the utilization of unsupervised learning in exploratory text mining tasks.

Tools and Libraries

- **Python 3.x** - for executing codes on Google Colab.
- **pandas & numpy** – for importing the dataset and managing the DataFrame objects.
- **matplotlib** – for importing the dataset and managing the DataFrame objects.
- **scikit-learn** – for vectorization by TF-IDF, K-Means, DBSCAN, and scaling, PCA.
- **google.colab.files** – for the transfer of files between notebook and local storage.
- **StandardScaler** – performing scaling on TF-IDF features before applying DBSCAN.

All development was done in **Google Colab**, which provided a simple environment for testing and visualizing the clustering steps.

Dataset Description

For this lab, I worked with cleaned_comments.csv, a file generated in Lab 2 after the text cleaning and preprocessing were performed. The file comprises the cleaned version of YouTube comments, in which the eliminating elements such as emojis, repeated characters, and formats that were extra and so on were done beforehand. In the dataset, there is a **`cleaned_tokens`** column, which is a list of tokens processed of every comment instead of raw text. The first step in the clustering task was to load the dataset into a pandas DataFrame in Google Colab and to ask about its structure by means of **`df.shape`** and **`df.head().`** This not only helped to confirm the total number of comments. The shown output signified that the notebook had already defined the **`cleaned_tokens`** column as the one containing the text to be worked with, which was then converted into a text corpus for TF-IDF vectorization. Thus, the dataset gave a very clean and standard collection of tokens to work with, which made it easy to apply clustering algorithms in later steps of the lab.

Methodology

1. Data Loading & Validation

The initial step of the analysis was to upload the **`cleaned_comments.csv file`** into Google Colab utilizing the file upload tool. After the file was read into a panda DataFrame, by **`df.shape`** checking the number of rows and columns and briefly inspecting the first few lines with **`df.head().`** The dataset was loaded accurately, and the **`cleaned_tokens`** column was at hand for utilization. The notebook also displayed which column would be utilized as the primary text source, that was a good measure to ensure the right data was chosen for further processing.

2. Text Vectorization (TF-IDF)

Clustering algorithms cannot use raw text as input, hence it was necessary to transform the cleaned comments into numerical features first. I opted for TF-IDF vectorization, which turns every comment into a weighted vector according to the relevance of the words in the entire dataset. The TF-IDF model was applied to the cleaned_tokens column [TF-IDF shape: (963,

2000)]. The result was a sparse matrix that signifies the vital word patterns and relationships, so it was the foundation of both clustering algorithms.

3. K-Means Clustering

a. Elbow Method

To determine the best number of clusters (k) to be used with K-Means, an elbow graph was created by running K-Means on k values from 2 to 7. To locate precisely the region where the rate of decline became quite steeply less, the inertia values (the measure of the quality of clustering in terms of these groups) were depicted. This "elbow point" served as a good warning for the number of clusters' selection without the danger of overfitting.

b. K-Means

Once optimal k was found using the elbow method, the K-Means algorithm was implemented on the TF-IDF vectors once again as the next step. The model then could classify every comment into a particular group, and this data was appended to the DataFrame as an additional column. Therefore, it was possible for me to find out the distribution of the comments across the various clusters and know the size of each cluster in terms of comments.

c. Interpretation

For K-Means output interpretation, I selected the most representative words from the centroid of each cluster. These words were crucial in identifying the topic or rather the main theme of each cluster. By comparing the most significant words, I was able to assume the nature of the comments in the clusters being together, although some clusters were more interpretable than others, depending on the dataset.

4. DBSCAN Clustering

a. Parameter Testing

For DBSCAN, parameter selection like eps (radius) and min_samples (neighbors) considerably impacts the outcome of clustering. The model was developed based on 3 predefined values for eps which were 0.3, 0.5, and 0.7 respectively. The numbers of clusters and noise comments obtained for every value were documented.

b. DBSCAN Visualization

After picking the eps that is optimal cluster structure, I re-ran DBSCAN. The results were visualized against the PCA-reduced components from earlier. The scatterplot revealed how

DBSCAN had grouped points according to their density and at the same time, it also indicated the comments which were marked as outliers (identified as -1). This representation allowed us to compare DBSCAN and K-Means more clearly as to the way they treated the data.

c. Interpretation

DBSCAN does not use every point in a cluster, so it clearly marks the noise comments that don't fit into any dense area. After observing each DBSCAN cluster by reviewing sample comments and got the number of comments which were classified as noise. Depending on the specific dataset some clusters were small and specific while others got very few or no points at all if the density was above threshold.

5. Comparison of Algorithms

1. Interpretability

- K-Means is more interpretable since it indicates the cluster centers plainly and keeps the groups stable.
- DBSCAN is less interpretable as the clusters are based on the density that might lead to the formation of shapes that are not regular.

2. Number of Clusters

- For K-means we must select the number of clusters (k) every time.
- DBSCAN creates the number of clusters that the data density can support, which might be more or less than the user anticipated.

3. Noise Handling

- K-Means places every comment into a cluster even if does not fit in the cluster.
- DBSCAN recognizes outliers as noise and does not put them into any clusters.

4. Insight

- K-Means delivered larger, more comprehensive themes that were more convenient to summarize.
- DBSCAN worked well in recognizing weird or clusters of comments but only when the density structure provided support.

Results and Discussion

1. Elbow Plot

The elbow plot was created for the range of k from **2 to 7** to determine the proper number of clusters for the K-Means algorithm. The graph showed a quick drop in inertia from **k = 2 to k = 3** and then, for larger k values, a less emphasized decrease. Once **k = 3** was reached, the slope of the line got less steep, meaning that the extra clusters would not be affecting compactness that much. Therefore, **k = 3** was selected as the optimal number of clusters.

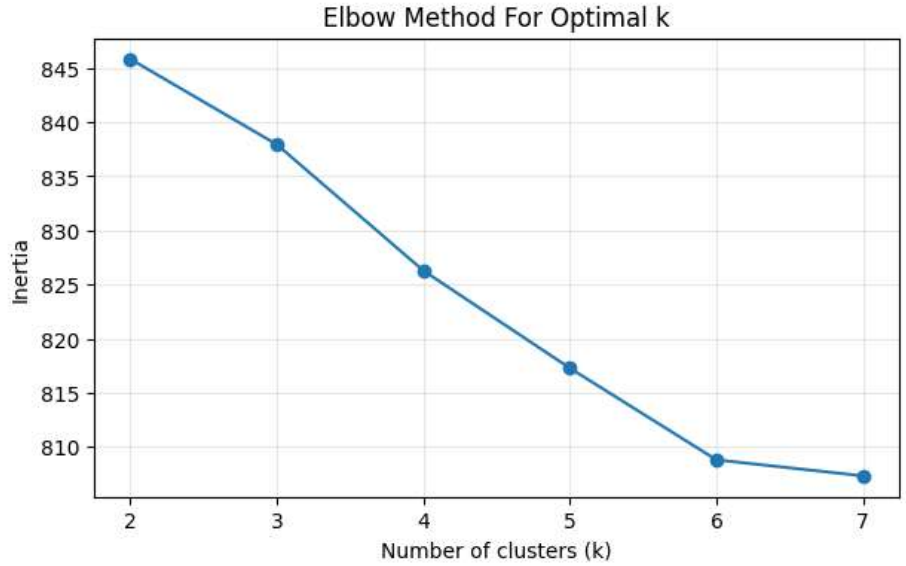

Figure 1. Elbow Method for Selecting Optimal k

2. K-Means Cluster Keywords

The K-Means model was implemented with **k = 3**, hence, every single comment was assigned to a particular cluster out of the three. As a further step, the top TF-IDF terms for each of the three clusters were utilized for the purpose of shedding light on the topics encapsulated in the dataset:

Cluster	Theme Description	Top Keywords
Cluster 0 (708 comments)	General positive engagement / appreciation	thank, video, thanks, quot, spotify, like, great, token, beast, amazing
Cluster 1 (152 comments)	Technical or course-related discussion	api, course, video, thank, best, good, spotify, really, great, apis
Cluster 2 (103 comments)	URL fragments / metadata noise	zgxjrw, wxsd, http, amp, com, www, href, youtube, watch, quot

Table 1. Top TF-IDF Keywords for Each K-Means Cluster

3. PCA Visualization for K-Means

The PCA plot for **k = 3** illustrates the spatial arrangement of TF-IDF vectors that represent the YouTube comments in a two-dimensional space. The comments on YouTube, being mainly short and with a very limited vocabulary, are the main factor for most of the points being tightly packed around the origin and thus, forming a large and very dense cluster.

Cluster 0 occupies the most significant area, and the theme of general positive engagement comments is even more distinct since these comments are made up of simple synonymous words such as "**thank**," "**video**," and "**like**." **Cluster 1** shows a slightly different and scattered distribution, which is associated with a more technical vocabulary that is connected to APIs and course discussions. **Cluster 2** is a tiny but distinctly identified group since it mainly contains URL fragments and metadata tokens that are extremely different from the average comment text. According to the PCA picture, the three clusters are quite separated although there is some overlap due to the compactness of the comments.

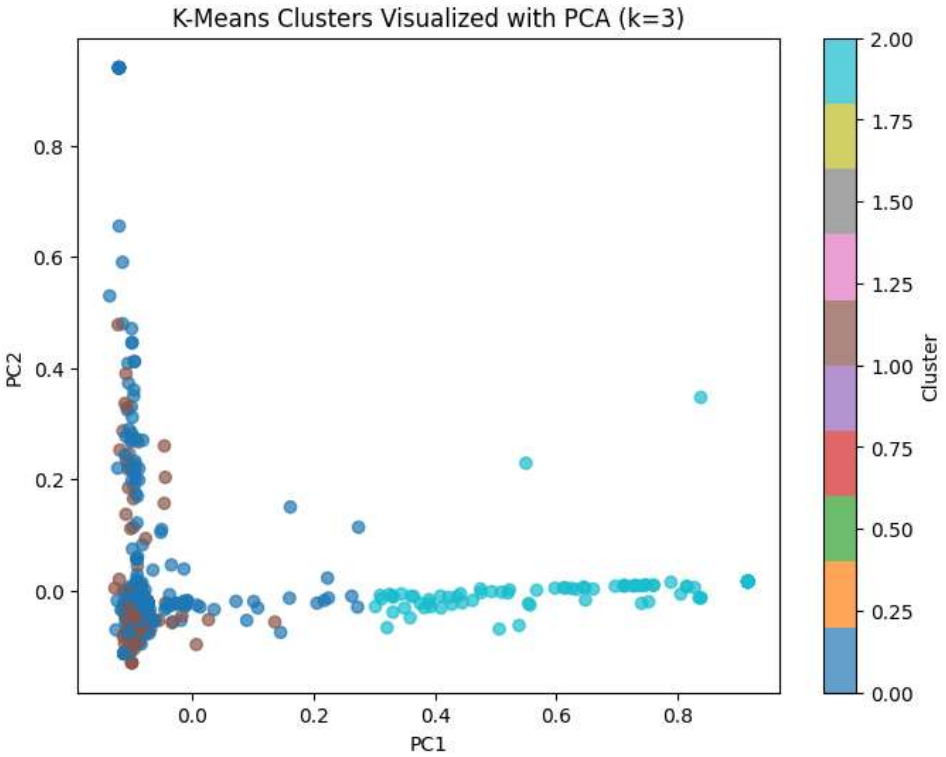

Figure 2. PCA Visualization of K-Means Clusters

4. DBSCAN Results

DBSCAN was applied with three different eps parameters (0.3, 0.5, and 0.7) while keeping min_samples at 5. In all situations, DBSCAN consistently delineated 4 clusters but additionally classified 867 comments as noise. Given that the dataset consists of around 963 comments total, this implies that most of the points were situated outside any dense region.

eps	min_samples	clusters	noise_points
0.3	5	4	867
0.5	5	4	867
0.7	5	4	867

Table 2. DBSCAN Parameter Testing Results

Based on these results, the visualization with eps = 0.5 was made. The PCA plot shows that the points were mainly classified as noise (**-1**), with just a very tiny cluster or two appearing in isolated places. The clusters consisted mostly of repetitive phrases (like "**thank you**") and URL tokens from YouTube, not discourse topics at all. This indicates that DBSCAN struggled to uncover any meaningful structure from the TF-IDF vectors of the YouTube short comments probably due to their sparse and high-dimensional nature.

5. Comparison of Algorithms

1. Interpretability

The three clusters of K-Means have well-defined, recognizable themes: general positive feedback, technical (**API**) discussion, and a (**URL**) noise cluster. On the other hand, DBSCAN grouped the whole dataset into one extremely large noise group with label **-1** and only a few tiny clusters like the cluster formed by the YouTube link tokens (**label 0**) and small groups of solicited "**Thank you**" or "**Thanks**" comments (**labels 2** and **3**). These DBSCAN clusters were less interpretable as reflecting meaningful topics.

2. Number of Clusters

With **k=3**, K-Means was able to give three different stable groups containing all the comments However, DBSCAN with the parameters **eps=0.5** and **min_samples=5** produced **4** small clusters and at the same time marked **867 comments** out of the total **963** as **noise** which meant that the algorithm had not assigned a big part of the data to any cluster thus implying that the main part of the dataset was not clustered.

3. Noise Handling

DBSCAN does highlight classifying the outliers as noise, yet its severity in this dataset was too high: most comments were classified as noise due to their very sparse and less packed TF-IDF vectors as for short comments. K-Means does not bin noise; it places every comment into a cluster, which facilitated the analyst's handling of the complete dataset.

Critical Prompts Answers

1. Initial hypothesis about cluster quality

The comments on YouTube are typically short, repetitive and written in a casual manner, so I expected the clusters to be quite broad before running the models. I thought that the clusters might not be very clear-cut, but I still expected to see some common themes like positive reactions or technical discussions at least.

2. Choice of k and justification

The elbow plot demonstrated a notable reduction in inertia from k = 2 to k = 3, and then the curve began to flatten. There was no significant improvement after k = 3, I selected k = 3 as the most logical choice.

3. K-Means vs DBSCAN: which was more useful and why?

K-Means proved to be considerably more beneficial for this dataset. It effectively organized comments into insightful themes and made the clusters quite interpretable. DBSCAN, on the other hand, faced difficulties as the TF-IDF vectors because the brief comments were extremely sparse, leading to most points being classified as noise. DBSCAN did not identify distinct subjects rather it mainly found repetitive patterns and a few URL-based bits. All in all, K-Means was the drawing that delivered the brightest insights

4. How dataset characteristics influenced the results and what this means for real applications

The comments in the dataset were short and simple, and had a limited vocabulary, which resulted in sparse feature vectors. This situation made density-based clustering hard, since DBSCAN had nothing to depend on regarding solid “dense areas”. This is quite a scenario in text mining where preprocessing and the choice of algorithms are very vital. For short social-media text, methods such as K-Means usually perform better than density-based approaches.

Conclusion

The lab allowed me to examine the performance of various clustering techniques on short, non-standard YouTube comments. After performing TF-IDF conversion of the purified texts, K-Means managed to generate three distinctly identifiable groups: comments expressing general satisfaction, discussions regarding technical/API matters, and a cluster where URLs-style tokens dominated the content. On the other hand, DBSCAN faced difficulties due to the text data's sparsity and hence, it considered most comments as noise, leading to the formation of only a few small and redundant clusters. Thus, this comparison reinforced the idea that data structure is a key factor in determining the superior clustering method. K-Means for short online comments revealed more consistent and fruitful patterns whereas DBSCAN was more of a tool for outlier detection rather than theme discovery. In conclusion, the lab provided me with an opportunity to get a grip on both algorithms' weaknesses and strengths, as well as the importance of preprocessing and model selection in practical text-mining tasks.